

Camadas LLM e Tools: Raciocínio, Seleção de Ferramentas e Efeito Real no Mundo & Skills, Subagentes e MCP: Orquestrando a Tripulação Agêntica — Um Recorte de AI Driven Development: Do Zero ao Deploy

Heverton Eduardo Peres

RESUMO

Este artigo investiga a articulação entre três camadas que transformam um modelo de linguagem de grande porte (LLM) em um agente de codificação capaz de produzir efeito real sobre repositórios e ambientes de produção: o raciocínio e a seleção de ferramentas do modelo, a orquestração via skills, subagentes e servidores do Model Context Protocol (MCP), e o contrato de configuração entre humano e agente materializado em arquivos como CLAUDE.md e AGENTS.md. A partir de um recorte temático do dossiê técnico consolidado sobre desenvolvimento orientado por inteligência artificial (2024-2026), o artigo sintetiza como cada camada amplia o raio de impacto potencial de uma decisão do modelo e identifica vetores de risco documentados na literatura de segurança recente, notadamente o tool poisoning em servidores MCP e a seleção adversarial de ferramentas. Os resultados indicam que nenhuma das três camadas, isoladamente, garante confiabilidade suficiente para dispensar supervisão humana em ações consequentes, e que qualquer política de governança de agentes de codificação precisa endereçar as três camadas simultaneamente: rigor documental na descrição de ferramentas, auditoria de skills e servidores MCP como superfície de ataque, e concisão nos arquivos de configuração de projeto. Conclui-se que a confiabilidade de um agente de codificação é uma propriedade emergente da arquitetura que o envolve, não apenas do modelo em si.

Palavras-chave: agentes de codificação, Model Context Protocol, seleção de ferramentas, engenharia de prompt, segurança de agentes de IA.

ABSTRACT

This article investigates the interplay between three layers that turn a large language model (LLM) into a coding agent capable of producing real effects on repositories and production environments: the model's reasoning and tool-selection subsystem, orchestration via skills, subagents, and Model Context Protocol (MCP) servers, and the human-agent configuration contract embodied in files such as CLAUDE.md and AGENTS.md. Drawing on a thematic excerpt of a consolidated technical dossier on AI-driven software development (2024-2026), the article synthesizes how each layer expands the potential blast radius of a model decision and identifies risk vectors documented in recent security literature, notably tool poisoning in MCP

servers and adversarial tool-selection attacks. Findings indicate that no single layer, in isolation, guarantees sufficient reliability to dispense with human oversight over consequential actions, and that any governance policy for coding agents must address all three layers simultaneously: documentation rigor for tool descriptions, security auditing of skills and MCP servers as an attack surface, and concise project configuration files. The article concludes that the reliability of a coding agent is an emergent property of the architecture surrounding it, not of the model alone.

Keywords: coding agents, Model Context Protocol, tool selection, prompt engineering, AI agent security.

1 INTRODUÇÃO

O desenvolvimento de software orientado por inteligência artificial migrou, entre 2024 e 2026, de um paradigma de autocompletar assistido para um paradigma de agência plena, no qual sistemas baseados em modelos de linguagem de grande porte (LLMs) planejam, selecionam ferramentas e produzem efeitos diretos sobre repositórios, pipelines e ambientes de produção. Dados de mercado recentes indicam adoção ativa por parcela majoritária das organizações de desenvolvimento de software (FUTURUM GROUP, 2026), e a Forrester descreve a migração de assistentes pontuais para agentes orquestradores de ciclo de vida completo como tendência dominante do período (FORRESTER, 2026). Esse deslocamento reabre, contudo, uma pergunta estrutural pouco discutida fora dos manuais de engenharia: o que exatamente acontece entre o instante em que um LLM recebe uma instrução e o instante em que uma ação concreta — escrever um arquivo, chamar uma API, abrir um pull request — é executada no mundo real?

A literatura técnica recente responde a essa pergunta com uma arquitetura em camadas. Anthropic e observadores independentes descrevem o harness — o runtime agêntico que envolve o modelo — como a camada que decide o que é permitido, enquanto o próprio modelo decide apenas o que tentar (PILLITTERI, 2026; MINDSTUDIO, 2026; AIMULTIPLE, 2026). Entre a intenção do modelo e a execução efetiva medeiam mecanismos de raciocínio estruturado — chain-of-thought, ReAct, Tree of Thoughts e Reflexion (IBM, 2026; PROMPTING, 2026; PROMPTHUB, 2026) —, um subsistema de seleção e chamada de ferramentas (function calling) com validação de esquema (HARTENFELLER, 2026; BLAXEL, 2026; AGENTA, 2026; PROMPTLAYER, 2025) e uma camada de composição multiagente que distribui trabalho entre skills, subagentes e servidores do Model Context Protocol (MCP) (SKILLS, 2026; ORCHESTRATE, 2026; MCP, 2026; WIKIPEDIA, 2026). Cada uma dessas camadas amplia o “raio de impacto” (blast radius) potencial de uma decisão tomada por um modelo estatístico, o que desloca o problema de engenharia de “o modelo acerta a resposta?” para “o sistema que

envolve o modelo contém o erro antes que ele produza efeito irreversível?” (SENTRY, 2026; OWASP, 2026).

Paralelamente, a camada de contrato entre humano e agente — arquivos como CLAUDE.md e AGENTS.md, que fixam contexto e regras específicas de projeto, e a engenharia de prompt propriamente dita — assumiu papel de especificação operacional, não de sugestão estilística (DEPLOYHQ, 2026; HUMANLAYER, 2025; TEAM400, 2026). Pesquisas sobre limites de seguimento de instrução em LLMs frontier sugerem que modelos seguem de forma confiável um número finito de diretrizes simultâneas, o que impõe restrições objetivas sobre quanto desses arquivos de configuração pode crescer antes de gerar comportamento imprevisível (MODIFYING, 2026; HOOKS, 2026).

1.1 Problema de Pesquisa e Objetivo

O problema de pesquisa deste recorte pode ser formulado assim: como as três camadas — raciocínio e seleção de ferramentas do LLM, orquestração via skills/subagentes/MCP, e o contrato de configuração humano-agente — se articulam para produzir (ou falhar em produzir) efeito confiável no mundo real, e quais vulnerabilidades estruturais essa articulação introduz? O objetivo deste artigo é sintetizar, a partir do dossiê técnico consolidado sobre desenvolvimento orientado por IA, uma leitura integrada dessas três camadas, evidenciando tanto os mecanismos que as tornam produtivas (TOTALUM, 2026; KONISHI, 2026; SUBAGENT, 2026) quanto os vetores de risco documentados na literatura de segurança de 2026, notadamente o tool poisoning em servidores MCP (OWASP, 2026; APTIBLE, 2026; WILLISON, 2025; CLOUD, 2026; SYSTEMATIZATION, 2025) e os ataques de seleção adversarial de ferramentas (TOOLTWEAK, 2025; BRIDGING, 2025).

1.2 Justificativa

A justificativa deste recorte é dupla. Em primeiro lugar, a maior parte da literatura de mercado trata “agentes de codificação” como uma caixa-preta unificada, obscurecendo o fato de que raciocínio, seleção de ferramentas e orquestração multiagente são subsistemas com falhas e garantias distintas — um agente pode raciocinar corretamente e ainda assim selecionar a ferramenta errada, ou selecionar a ferramenta certa e ainda assim ser vítima de uma descrição de ferramenta envenenada (SENTRY, 2026; OWASP, 2026). Em segundo lugar, práticas corporativas documentadas de integração de agentes em pipelines de CI/CD dependem, na prática, exatamente dessa separação de camadas para manter um portão de aprovação humana antes do deploy em produção (GOVERNED, 2026; GITINJECT, 2026), o que torna a compreensão precisa de cada camada uma condição prévia para qualquer política de governança de risco aplicada a esses sistemas. O restante do artigo detalha, na Seção 2, como o recorte foi construído a partir do dossiê-mãe; na Seção 3, discute em profundidade as três camadas e suas interseções; e na Seção 4, sintetiza as implicações práticas.

2 METODOLOGIA

Este artigo não reporta um experimento controlado nem coleta dados primários; trata-se de um recorte investigativo de natureza documental-analítica, construído por reaproveitamento de um dossiê técnico já indexado, consolidado previamente sobre desenvolvimento de software orientado por IA. O método, portanto, não é o de validação empírica de hipótese, mas o de síntese estruturada de literatura técnica e científica já mineirada, organizada segundo um recorte temático específico dentro de uma obra-mãe mais ampla.

2.1 Corpus e Fonte de Dados

O corpus deste recorte corresponde aos blocos de conteúdo indexados nos capítulos 4, 5 e 6 do sumário macro da obra-mãe “AI Driven Development: Do Zero ao Deploy”, que tratam, respectivamente, da arquitetura em quatro camadas (Tela, Harness, LLM, Tools) (MINDSTUDIO, 2026; PILLITTERI, 2026; AIMULTIPLE, 2026), da configuração de skills, subagentes e MCP (SKILLS, 2026; ORCHESTRATE, 2026; MCP, 2026; TOTALUM, 2026; KONISHI, 2026) e da engenharia de prompt e dos arquivos de contrato CLAUDE.md/AGENTS.md (DEPLOYHQ, 2026; HUMANLAYER, 2025; TEAM400, 2026; MODIFYING, 2026). Esse dossiê consolida mais de oitenta fontes brutas verificáveis — documentação oficial de fornecedores, artigos de pesquisa em repositórios de pré-publicação e relatos técnicos de adoção corporativa —, das quais este recorte seleciona e reorganiza o subconjunto pertinente aos três pilares enunciados na introdução.

2.2 Critério de Seleção e Recorte

O critério de seleção de blocos obedeceu a relevância temática recuperada por indexação semântica local (TF-IDF) sobre o dossiê-mãe, consultado por meio de buscas dirigidas aos três pilares: (i) raciocínio do LLM e seleção de ferramentas com efeito real no mundo (HARTENFELLER, 2026; BLAXEL, 2026; SENTRY, 2026; AGENTA, 2026; PROMPTLAYER, 2025; IBM, 2026; PROMPTING, 2026; PROMPTHUB, 2026; COMET, 2026); (ii) orquestração via skills, subagentes e MCP (WIKIPEDIA, 2026; WEBFUSE, 2026; SUBAGENT, 2026); e (iii) o contrato de configuração entre humano e agente via CLAUDE.md/AGENTS.md e engenharia de prompt (HOOKS, 2026; HUMANLAYER, 2026). Nenhuma fonte nova foi pesquisada para a produção deste artigo — a regra de reaproveitamento integral do dossiê-mãe é deliberada, e visa preservar a rastreabilidade entre a obra completa e cada recorte derivado, evitando divergência factual entre livro-mãe e artigos.

2.3 Procedimento de Síntese

A partir dos blocos recuperados, o procedimento de síntese seguiu três etapas: primeiro, agrupamento dos blocos por pilar temático; segundo, identificação de convergências e tensões entre fontes (por exemplo, entre a promessa de autonomia de orquestração multiagente e os riscos

de segurança documentados para MCP) (OWASP, 2026; APTIBLE, 2026; WILLISON, 2025; CLOUD, 2026; GENERAL, 2026; SYSTEMATIZATION, 2025); terceiro, redação impessoal em terceira pessoa com citação autor-data obrigatória para toda afirmação factual, seguindo a norma NBR 10520, e numeração progressiva de seções conforme a NBR 6024. Vetores de risco documentados em pesquisa recente sobre ataques de seleção de ferramentas (TOOLTWEAK, 2025; BRIDGING, 2025) e sobre injeção de prompt em pipelines de CI/CD (GITINJECT, 2026) foram tratados como achados de igual peso analítico aos relatos de adoção corporativa (FORRESTER, 2026; FUTURUM GROUP, 2026), evitando o viés de otimismo tecnológico comum em material de marketing de fornecedores.

2.4 Limitações do Método

Por depender de um dossiê já constituído, este recorte herda as limitações da pesquisa original: predominância de fontes de documentação técnica de fornecedores e de blogs especializados sobre artigos revisados por pares, e uma janela temporal concentrada em 2024–2026, período de mudança acelerada nas práticas de mercado (GOVERNED, 2026). A ausência de coleta primária impede qualquer inferência causal sobre eficácia comparada entre arquiteturas de orquestração (LangGraph, CrewAI, AutoGen ou Dynamic Workflows); o artigo limita-se a mapear o estado documentado da técnica e suas tensões internas, não a medi-lo empiricamente (CODEANT, 2026; YAGE, 2026; LUHARUKA, 2026; REDIS, 2026; REDIS, 2025).

3 RESULTADOS E DISCUSSÃO

A síntese do dossiê-mãe aplicada a este recorte permite organizar os achados em três blocos que correspondem aos pilares anunciados na introdução: o raciocínio do LLM e a seleção de ferramentas com efeito real no mundo; a orquestração via skills, subagentes e MCP; e o contrato de configuração entre humano e agente.

3.1 Raciocínio, Seleção de Ferramentas e Efeito Real no Mundo

Anthropic descreve a construção de agentes eficazes como um problema de composição de blocos simples e testáveis, e não de arquiteturas monolíticas complexas (ANTHROPIC, 2024). Estruturas de raciocínio como chain-of-thought guiam o modelo por um processo de decomposição passo a passo antes de agir (IBM, 2026), enquanto ReAct, Tree of Thoughts e Reflexion fornecem o andaime que transforma um modelo capaz em um agente confiável, capaz de revisar sua própria trajetória de decisão (PROMPTING, 2026; PROMPTHUB, 2026; COMET, 2026). Esse raciocínio, porém, só produz efeito no mundo real quando acoplado a um mecanismo de chamada de função (function calling): a documentação técnica converge no ponto de que ferramentas devem ser tratadas como parte do prompt, com o mesmo esforço de especificação dedicado à redação do prompt em si — nome, descrição e esquema de cada ferramenta influenciam diretamente a taxa de acerto de seleção (HARTENFELLER, 2026; BLAXEL, 2026).

A saída estruturada (structured output), forçada por esquema JSON, emerge como prática obrigatória para tornar o parsing da decisão do modelo confiável (AGENTA, 2026; PROMPTLAYER, 2025), e a chamada de função programática — em que o modelo gera código que invoca ferramentas em vez de emitir uma única chamada por turno — reduz o número de idas e vindas entre modelo e ambiente de execução (PROGRAMMATIC, 2026). O ponto de maior tensão nesta camada, contudo, é de segurança: validação de toda saída de chamada de função antes da execução e checagem de esquema são práticas obrigatórias, não opcionais, porque operações sensíveis não podem depender exclusivamente do raciocínio do LLM para se autolimitar — exigem regras de validação determinísticas independentes (SENTRY, 2026). Pesquisa recente documenta ataques de manipulação da seleção de ferramentas, em que a ordem, nomenclatura ou descrição das ferramentas disponíveis é explorada para induzir o agente a escolher uma ferramenta diferente da pretendida (TOOLTWEAK, 2025), e uma avaliação comparativa de vulnerabilidade em paradigmas de implantação de agentes LLM aponta que o “efeito real no mundo” de uma chamada de ferramenta — gravar um arquivo, disparar um deploy, mover dinheiro — é exatamente o que torna esse subsistema o de maior custo de falha de toda a arquitetura (BRIDGING, 2025).

3.2 Skills, Subagentes e MCP: a Camada de Orquestração

A camada de orquestração resolve um problema distinto: como distribuir trabalho entre unidades especializadas sem que cada uma precise reconstruir o contexto do zero. Skills empacotam instruções, metadados e recursos opcionais que o modelo invoca automaticamente quando relevante (SKILLS, 2026), enquanto subagentes são instâncias isoladas disparadas pela sessão principal para trabalhar em paralelo, cada uma com sua própria janela de contexto, permissões e modelo — a propriedade definidora de um subagente é justamente começar com contexto limpo, sem acesso ao histórico da thread-mãe (TOTALUM, 2026; KONISHI, 2026). Em 2026, esse padrão evoluiu para *dynamic workflows*, nos quais um agente líder pode planejar e disparar dezenas a centenas de subagentes paralelos em uma única sessão, avaliados por um sistema separado de aferição de desempenho (ORCHESTRATE, 2026). Frameworks de orquestração mais amplos — LangGraph, CrewAI, AutoGen — formalizam padrões equivalentes de encadeamento de prompt, roteamento e composição orquestrador-trabalhadores (SUBAGENT, 2026).

O Model Context Protocol (MCP) é o padrão que amarra essa camada às ferramentas e fontes de dados externas, substituindo integrações fragmentadas por um protocolo único (MCP, 2026; WIKIPEDIA, 2026; WEBFUSE, 2026). Sua neutralidade em relação a fornecedor — reforçada pela doação do protocolo a uma fundação neutra sob a Linux Foundation — não elimina, contudo, um vetor de risco documentado extensivamente pela comunidade de segurança: o *tool poisoning*, em que instruções maliciosas são embutidas diretamente na descrição de uma ferramenta MCP e injetadas no contexto do LLM durante o registro, sem se assemelhar à injeção de prompt tradicional (OWASP, 2026). Esse achado é corroborado por múltiplas fontes independentes: análises de “raio de impacto” (blast radius) de servidores MCP comprometidos (APTIBLE, 2026), o relato pioneiro de Simon Willison sobre problemas estruturais de segurança

do protocolo (WILLISON, 2025), um guia de boas práticas de segurança agêntica em MCP produzido pela Cloud Security Alliance (CLOUD, 2026) e uma sistematização acadêmica de conhecimento sobre segurança e salvaguardas no ecossistema MCP (SYSTEMATIZATION, 2025). A convergência dessas fontes sugere que a camada de orquestração, por multiplicar o número de pontos de entrada de ferramentas de terceiros, multiplica proporcionalmente a superfície de ataque disponível a um agente autônomo.

3.3 CLAUDE.md, AGENTS.md e o Contrato de Configuração

A terceira camada regula o que o agente pode e deve fazer dentro de um projeto específico. Arquivos CLAUDE.md fornecem contexto e instruções de projeto, mas exigem configuração explícita para ser carregados automaticamente; AGENTS.md funciona como padrão de fallback multi-ferramenta, permitindo que uma única especificação sirva a diferentes IDEs e CLIs agênticas (DEPLOYHQ, 2026). A convergência prática recomendada por múltiplas fontes é manter esses arquivos concisos — abaixo de algumas centenas de linhas —, porque a capacidade de seguimento confiável de instruções de um LLM frontier é finita, e o próprio prompt de sistema do harness já consome parcela relevante desse orçamento (HUMANLAYER, 2025; HOOKS, 2026; MODIFYING, 2026).

A engenharia de prompt para agentes de código, nesse contexto, deixou de ser apenas escolha de palavras: a Anthropic descreve a evolução necessária para *context engineering* — o conjunto de estratégias para curar e manter o conjunto ótimo de tokens durante a inferência, incluindo toda informação que chega à janela de contexto fora do prompt propriamente dito (ANTHROPIC, 2025). Um guia complementar sobre a construção de harnesses eficazes para agentes de longa duração recomenda estruturas de prompt diferenciadas entre a primeiríssima janela de contexto e janelas subsequentes em fluxos multi-janela (ANTHROPIC, 2026). Técnicas de gestão de comprimento de contexto — sumarização incremental, poda de histórico, recuperação seletiva — aparecem como resposta direta ao problema de *overflow* de janela em sessões longas (REDIS, 2026; REDIS, 2025; LUHARUKA, 2026), e a preferência documentada de agentes de codificação por ferramentas de busca leves como grep/ripgrep, em vez de indexação semântica pesada, ilustra como a economia de tokens se tornou um critério de design tão relevante quanto a precisão de busca (CODEANT, 2026; YAGE, 2026).

3.4 Discussão Integrada: onde as Três Camadas se Tocam

O achado transversal deste recorte é que as três camadas não operam isoladamente: uma configuração de CLAUDE.md mal escrita pode induzir o LLM a selecionar a ferramenta errada mesmo com raciocínio correto; uma skill mal documentada equivale, na prática, a uma ferramenta com esquema ambíguo, reproduzindo o mesmo risco de seleção incorreta descrito para function calling (TOOLTWEAK, 2025); e um servidor MCP comprometido explora exatamente a confiança que a camada de orquestração deposita nas descrições de ferramenta como parte do contrato entre humano e agente (OWASP, 2026; GITINJECT, 2026). Casos corporativos documentados de integração de agentes em CI/CD — revisão de pull request,

reparo de testes, triagem de falhas de build e remediação de segurança — mantêm, em todos os relatos revisados, um portão de aprovação humana antes do deploy em produção precisamente porque nenhuma das três camadas, isoladamente, oferece garantia suficiente de segurança para dispensar essa checagem (GOVERNED, 2026; FORRESTER, 2026). Ferramentas concorrentes como o GitHub Copilot em modo agente adotam a mesma lógica de escalonamento de decisões consequentes ao humano (GITHUB, 2025; VISUAL STUDIO CODE, 2025), reforçando que a tensão entre autonomia e controle não é peculiaridade de uma única plataforma, mas uma característica estrutural da arquitetura de três camadas aqui descrita.

4 CONCLUSÃO

Este recorte demonstrou que a arquitetura que transforma um LLM em um agente de codificação capaz de produzir efeito real no mundo não é monolítica, mas estratificada em três camadas com propriedades e falhas distintas: o raciocínio e a seleção de ferramentas do modelo (HARTENFELLER, 2026; SENTRY, 2026; TOOLTWEAK, 2025), a orquestração via skills, subagentes e servidores MCP (SKILLS, 2026; TOTALUM, 2026; OWASP, 2026), e o contrato de configuração entre humano e agente materializado em CLAUDE.md, AGENTS.md e engenharia de prompt (DEPLOYHQ, 2026; ANTHROPIC, 2025). A convergência de múltiplas fontes independentes — documentação de fornecedor, pesquisa em segurança e relatos corporativos de adoção — indica que nenhuma dessas camadas, isoladamente, garante confiabilidade suficiente para dispensar supervisão humana em ações consequentes (GOVERNED, 2026; GITINJECT, 2026; FORRESTER, 2026).

A implicação prática central é que qualquer política de governança de agentes de codificação precisa endereçar as três camadas simultaneamente: tratar a documentação de ferramentas com o mesmo rigor de um contrato de API (BLAXEL, 2026; AGENTA, 2026), auditar descrições de skills e servidores MCP como superfície de ataque (APTIBLE, 2026; CLOUD, 2026; SYSTEMATIZATION, 2025), e manter arquivos de configuração de projeto concisos o suficiente para não competir com o próprio orçamento de instruções do harness (HUMANLAYER, 2025; HOOKS, 2026). Trabalhos futuros, fora do escopo documental deste recorte, poderiam medir empiricamente a taxa de falha de seleção de ferramentas sob diferentes formatos de documentação, algo que a literatura consultada ainda trata majoritariamente em nível qualitativo (BRIDGING, 2025; WILLISON, 2025).

REFERÊNCIAS

AGENTA. *The guide to structured outputs and function calling with LLMs*. 2026. Disponível em: <https://agenta.ai/blog/the-guide-to-structured-outputs-and-function-calling-with-llms>. Acesso em: 02 ago. 2026.

AIMULTIPLE. *Top Agent Harnesses: Claude Code vs Codex*. 2026. Disponível em: <https://aimultiple.com/agent-harness>. Acesso em: 02 ago. 2026.

ANTHROPIC. *Building Effective AI Agents*. 2024. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. *Effective context engineering for AI agents*. 2025. Disponível em: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. *Effective harnesses for long-running agents*. 2026. Disponível em: <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>. Acesso em: 02 ago. 2026.

APTIBLE. *Prompt Injection in MCP: Tool Poisoning and Blast Radius*. 2026. Disponível em: <https://www.aptible.com/mcp-security/mcp-prompt-injection>. Acesso em: 02 ago. 2026.

BLAXEL. *What Is LLM Function Calling?*. 2026. Disponível em: <https://blaxel.ai/blog/what-is-llm-function-calling>. Acesso em: 02 ago. 2026.

BRIDGING AI and Software Security: A Comparative Vulnerability Assessment of LLM Agent Deployment Paradigms. ARXIV.ORG, 2025. Disponível em: <https://arxiv.org/pdf/2507.06323>. Acesso em: 02 ago. 2026.

CLOUD SECURITY ALLIANCE. *Agentic MCP Security Best Practices Guide*. 2026. Disponível em: <https://labs.cloudsecurityalliance.org/agentic/agentic-mcp-security-best-practices-v1/>. Acesso em: 02 ago. 2026.

CODEANT. *Why Your Coding Agent Should Use ripgrep (rg) Instead of grep*. 2026. Disponível em: <https://codeant.ai/blogs/why-coding-agents-should-use-ripgrep>. Acesso em: 02 ago. 2026.

COMET. *Prompt Engineering for Agentic AI Systems: An Introduction*. 2026. Disponível em: <https://www.comet.com/site/blog/prompt-engineering/>. Acesso em: 02 ago. 2026.

DEPLOYHQ. *CLAUDE.md, AGENTS.md & Copilot Instructions: Configure Every AI Coding Assistant*. 2026. Disponível em: <https://www.deployhq.com/blog/ai-coding-config-files-guide>. Acesso em: 02 ago. 2026.

FORRESTER. *Agentic Software Development Takes The Lead: From Code Assistants To Orchestrated SDLC Agents*. 2026. Disponível em: <https://www.forrester.com/blogs/agentic-software-development-takes-the-lead-from-code-assistants-to-orchestrated-sdlc-agents/>. Acesso em: 02 ago. 2026.

FUTURUM GROUP. *AI Reaches 97% of Software Development Organizations*. 2026. Disponível em: <https://futuraumgroup.com/press-release/ai-reaches-97-of-software-development-organizations/>. Acesso em: 02 ago. 2026.

GENERAL ANALYSIS. *Anthropic Claude Code Security Best Practices: Permissions, Hooks, MCP, Sandboxing, and CI/CD*. 2026. Disponível em: <https://generalanalysis.com/guides/anthropic-claude-code-security-best-practices>. Acesso em: 02 ago. 2026.

GITHUB. *Agent mode 101: All about GitHub Copilot's powerful mode*. 2025. Disponível em: <https://github.blog/ai-and-ml/github-copilot/agent-mode-101-all-about-github-copilots-powerful-mode/>. Acesso em: 02 ago. 2026.

GITINJECT: Real-World Prompt Injection Attacks in AI-Powered CI/CD Pipelines. ARXIV.ORG, 2026. Disponível em: <https://arxiv.org/pdf/2606.09935>. Acesso em: 02 ago. 2026.

GOVERNED AI-Assisted Engineering: Graduated Human Oversight for Agentic Code Generation in Regulated Domains. ARXIV.ORG, 2026. Disponível em: <https://arxiv.org/pdf/2606.22484>. Acesso em: 02 ago. 2026.

HARTENFELLER. *Best Practices for LLM Tools or Function Calling for Oracle Developers*. 2026. Disponível em: <https://hartenfeller.dev/blog/ai-tools-best-practice-oracle>. Acesso em: 02 ago. 2026.

HOOKS reference. ANTHROPIC, Claude Code Docs, 2026. Disponível em: <https://code.claude.com/docs/en/hooks>. Acesso em: 02 ago. 2026.

HUMANLAYER. *12-Factor Agents — Principles for Building Reliable LLM Applications*. 2026. Disponível em: <https://www.humanlayer.dev/12-factor-agents>. Acesso em: 02 ago. 2026.

HUMANLAYER. *Writing a good CLAUDE.md*. 2025. Disponível em: <https://www.humanlayer.dev/blog/writing-a-good-claude-md>. Acesso em: 02 ago. 2026.

IBM. *What is chain of thought (CoT) prompting?*. 2026. Disponível em: <https://www.ibm.com/think/topics/chain-of-thoughts>. Acesso em: 02 ago. 2026.

KONISHI, Hidekazu. *Claude Code Subagents and Multi-Agent Orchestration Guide*. 2026. Disponível em: https://hidekazu-konishi.com/entry/claude_code_subagents_and_orchestration_guide.html. Acesso em: 02 ago. 2026.

LUHARUKA, Shubham. *Context Optimization: A Comprehensive Framework for Reducing Large Language Model Token Usage*. 2026. Disponível em: <https://luharuka.medium.com/context-optimization-a-comprehensive-framework-for-reducing-large-language-model-token-usage-fed8d9229e30>. Acesso em: 02 ago. 2026.

MCP. *Specification and documentation for the Model Context Protocol*. 2026. Disponível em: <https://github.com/modelcontextprotocol/modelcontextprotocol>. Acesso em: 02 ago. 2026.

MINDSTUDIO. *What Is an Agent Harness? The Architecture Behind Claude Code, Codex, and Cursor*. 2026. Disponível em: <https://www.mindstudio.ai/blog/what-is-agent-harness-architecture-explained>. Acesso em: 02 ago. 2026.

MODIFYING system prompts. ANTHROPIC, Claude API Docs, 2026. Disponível em: <https://platform.claude.com/docs/en/agent-sdk/modifying-system-prompts>. Acesso em: 02 ago. 2026.

ORCHESTRATE subagents at scale with dynamic workflows. ANTHROPIC, Claude Code Docs, 2026. Disponível em: <https://code.claude.com/docs/en/workflows>. Acesso em: 02 ago. 2026.

OWASP FOUNDATION. *MCP Tool Poisoning*. 2026. Disponível em: https://owasp.org/www-community/attacks/MCP_Tool_Poisoning. Acesso em: 02 ago. 2026.

PILLITTERI, Pasquale. *Claude Code Harness: The Runtime Architecture That Turns an LLM into an Autonomous Agent*. 2026. Disponível em: <https://pasqualepillitteri.it/en/news/1892/claude-code-harness-runtime-architecture-2026-guide>. Acesso em: 02 ago. 2026.

PROGRAMMATIC tool calling. ANTHROPIC, Claude Platform Docs, 2026. Disponível em: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/programmatic-tool-calling>. Acesso em: 02 ago. 2026.

PROMPTHUB. *Prompt Engineering for AI Agents*. 2026. Disponível em: <https://www.prompthub.us/blog/prompt-engineering-for-ai-agents>. Acesso em: 02 ago. 2026.

PROMPTING GUIDE. *Tree of Thoughts (ToT)*. 2026. Disponível em: <https://www.promptingguide.ai/techniques/tot>. Acesso em: 02 ago. 2026.

PROMPTLAYER. *How JSON Schema Works for LLM Tools & Structured Outputs*. 2025. Disponível em: <https://blog.promptlayer.com/how-json-schema-works-for-structured-outputs-and-tool-integration/>. Acesso em: 02 ago. 2026.

REDIS. *Context Window Management for LLM Apps: Dev Guide*. 2025. Disponível em: <https://redis.io/blog/context-window-management-llm-apps-developer-guide/>. Acesso em: 02 ago. 2026.

REDIS. *Context Window Overflow in 2026: Fix LLM Errors Fast*. 2026. Disponível em: <https://redis.io/blog/context-window-overflow/>. Acesso em: 02 ago. 2026.

SENTRY. *Exploiting Tool and Function Calling in LLM Agents*. 2026. Disponível em: <https://blog.sentry.security/exploiting-tool-and-function-calling-in-llm-agents/>. Acesso em: 02 ago. 2026.

SKILLS. Agent Skills — Claude Platform Docs. ANTHROPIC, 2026. Disponível em: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>. Acesso em: 02 ago. 2026.

SUBAGENT Orchestration Guide — Claude Code Skill. MCP MARKET, 2026. Disponível em: <https://mcpmarket.com/tools/skills/subagent-orchestration-guide-1>. Acesso em: 02 ago. 2026.

SYSTEMATIZATION of Knowledge: Security and Safety in the Model Context Protocol Ecosystem. ARXIV.ORG, 2025. Disponível em: <https://arxiv.org/pdf/2512.08290>. Acesso em: 02 ago. 2026.

TEAM400. *Claude Agent SDK — How to Customise System Prompts for Your AI Agents*. 2026. Disponível em: <https://team400.ai/blog/2026-04-claude-agent-sdk-system-prompts-customisation>. Acesso em: 02 ago. 2026.

TOOLTWEAK: An Attack on Tool Selection in LLM-based Agents. ARXIV.ORG, 2025. Disponível em: <https://arxiv.org/pdf/2510.02554>. Acesso em: 02 ago. 2026.

TOTALUM. *Claude Code subagents: the 2026 production playbook*. 2026. Disponível em: <https://www.totalum.app/blog/claude-code-subagents-totalum>. Acesso em: 02 ago. 2026.

VISUAL STUDIO CODE. *Introducing GitHub Copilot agent mode (preview)*. 2025. Disponível em: <https://code.visualstudio.com/blogs/2025/02/24/introducing-copilot-agent-mode>. Acesso em: 02 ago. 2026.

WEBFUSE. *MCP Cheat Sheet: Model Context Protocol Quick Reference*. 2026. Disponível em: <https://www.webfuse.com/mcp-cheat-sheet>. Acesso em: 02 ago. 2026.

WIKIPEDIA. *Model Context Protocol*. 2026. Disponível em: https://en.wikipedia.org/wiki/Model_Context_Protocol. Acesso em: 02 ago. 2026.

WILLISON, Simon. *Model Context Protocol has prompt injection security problems*. 2025. Disponível em: <https://simonwillison.net/2025/Apr/9/mcp-prompt-injection/>. Acesso em: 02 ago. 2026.

YAGE.AI. *Why Coding Agents Still Use grep as Their Search Backbone*. 2026. Disponível em: <https://yage.ai/share/why-coding-agents-still-use-grep-en-20260327.html>. Acesso em: 02 ago. 2026.